

Block 11e: verified first-boot hardware entropy

Date: 2026-09-28. Auditor: Claude (Opus 5.5). Isolated checkout: /private/tmp/kk715-11e. Immediate target: fork audit/715-11d-future-storage-guards-20260928 at 0c08ce9ef4869269922c4c0b62f7ac069d7c81ea (PR #883). Carried unit content: commit 63789e539, the seven 11e paths unchanged from cumulative PR #877 head 5334f77c351373a900d67d908a467e5f3e08b551. Audit repair: commit 6fb90ed71. Final containing head and hosted CI belong in the PR body.

1. Scope and source reconciliation

`flash_collectHwEntropy()` runs once per boot in signed firmware, before privileges drop and before `kk_board_init()`. When the OTP randomness block is unlocked it draws 32 bytes, programs them, and locks the block forever; the bytes feed the PIN KDF salt through `flash_readHwEntropy()`. The carried unit (Copilot finding 4118361277) makes every failure halt before DRBG/storage can use erased or partial OTP bytes: a rejected draw, a rejected or unverified write, a failed lock, or a failed read of a locked block. Unsigned firmware keeps its documented fixed 0x3C entropy. The emulator branch is unchanged.

The seven paths match the split map exactly: `lib/board/keepkey_flash.c`, `unittests/board/CMakeLists.txt`, the two `hardware_stubs` headers, and `hw_entropy.cpp` / `hw_entropy_probe.{c,h}`. The probe compiles the real device branch of `keepkey_flash.c` on the host, with only register, OTP, RNG and halt dependencies substituted.

2. Audit finding: a halted first boot could never boot again

OTP programming only clears bits. The carried code halted when verification or the lock failed, but left the block unlocked with whatever bytes had been programmed. The next boot saw an unlocked block, drew *fresh* entropy, and programmed it over the non-erased bytes. The result is the AND of the two draws, so verification failed and the device halted again, on every later boot, for every signed firmware containing the guard. Triggers: power loss between programming and lock, a partially programmed block, or a lock that did not take effect. The bootloader is unaffected, so this is loss of the firmware rather than a hard brick, but no signed update could recover it.

The unit tests missed this because the probe modelled an OTP write as `memcpy` and every test booted exactly once.

Repair (6fb90ed71): on an unlocked block, read it first, keep bytes already programmed, and program only erased (0xFF) bytes, one byte at a time. Then verify the whole block, lock, and verify the lock as before. In the guarded interruption scenario, kept bytes came from this firmware's earlier checked draw. The code cannot prove that for arbitrary unlocked OTP: bytes written by older firmware, or by another privileged path, are kept unchecked. That adds no new trust: code able to program OTP before the lock already runs privileged, and could equally program and lock chosen bytes. The probe now ANDs bits per byte like the device, a reboot helper carries OTP and lock state across boots, and two tests cover recovery:

- `ProgrammedButUnlockedBlockIsKeptAndLocked`: a fully programmed, unlocked block is locked as is, with zero writes.
- `InterruptedFirstBootCompletesOnNextHealthyBoot`: after a partial write, a rejected lock, or a dropped lock halts the first boot, the next healthy boot completes and locks (16, 0 and 0 writes). The reboot draws different bytes, and the test asserts that programmed bytes keep the first draw while erased bytes take the second.

3. Verification

Local Docker is unresponsive on this host, so the full emulator unit suites and `cppcheck` could not run locally. They run in the hosted CI recorded in the PR (make `xunit` emits `board.xml`). The entropy tests were compiled natively from this checkout's pinned sources with `-Wall -Werror`

and linked against pinned googletest.

Local check	Result
HardwareEntropy suite at 380c75bc3 (fix 6fb90ed71)	9 of 9 pass
Negative control: same tests, unfixed keepkey_flash.c from 5334f77c3	4 fail: both recovery tests halt on the reboot (returned false, halted true), and the two updated write/read counts differ
clang-format 20 on changed files	Clean
Preflight (PREFLIGHT_BASE=0c08ce9ef)	All checks pass except cppcheck, which was not run (Docker unresponsive)

Evidence archive 715-11e-controls-20260928.tgz binds the fixed and control JUnit XML/logs and the preflight log. Physical-device behavior is not claimed. The recovery path is exercised only in the host probe; hardware OTP programming faults were not injected.

4. Hosted CI and Copilot round 1

Manual run

[36514449899](https://github.com/BitHighlander/keepkey-firmware/actions/runs/36514449899) passed every required job at 226f3d0ee. Artifacts were verified: HardwareEntropy 9/9 in both variants and both harnesses, and both ARM variants with 23 files. Aggregate: 1,560 passed, 84 declared skips, 0 failures.

Copilot review 5347263366 on 226f3d0ee required a test change, so round one fails. (1) Every boot drew identical bytes. The unfixed code then rewrote identical values on reboot, verified, and the recovery test failed only on its write count. The first control claim in this report was therefore overstated. Only the 0x19 case reproduced the halt. Reboots now draw different bytes, and the control halts. (2) The provenance claim above is narrowed to the guarded interruption scenario. (3) The adjacent inventory below was added. The firmware change is unchanged.

5. Adjacent inventory

Inventory base: 0c08ce9ef4869269922c4c0b62f7ac069d7c81ea. Reproduce with `git diff --numstat BASE..FINAL_PR_HEAD`.

Added	Deleted	Path
40	10	lib/board/keepkey_flash.c
6	0	unittests/board/CMakeLists.txt
5	0	unittests/board/hardware_stubs/libopencm3/stm32/desig.h
9	0	unittests/board/hardware_stubs/libopencm3/stm32/flash.h
143	0	unittests/board/hw_entropy.cpp
178	0	unittests/board/hw_entropy_probe.c
31	0	unittests/board/hw_entropy_probe.h
-	-	docs/release/audit-units/715-11e-controls-20260928.tgz
56	0	docs/release/audit-units/715-11e-hardware-entropy-20260928.md
-	-	docs/release/audit-units/715-11e-hardware-entropy-20260928.pdf